M T W T F S S
Page No
Date 30/12/25
YOUVA

Information Security

Lec-1

information = randomness = lack of knowledge

↓

independence

=

freedom

non random = determinism

Information theoretic security & Computational security

- information can not be determined by ANY COMPUTER.

eg $f(x) = y$ x & fnc^n ⟶ info^n

Lec-2 01/01/26

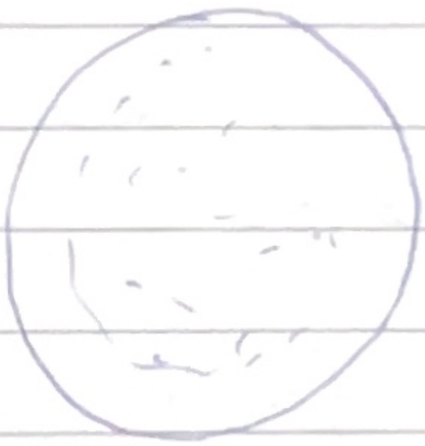
? Quantum Source

Lack of knowledge ⇒ randomness ⇒ independence ⇒ information
security free will

$A \Rightarrow B$
$\neg B \Rightarrow \neg A$

$B \Rightarrow A$

Classical view of universe



1. Elementary particles
2. Basic forces

→ Even with all knowledge this is still random (Quantum source)

"The Universe is not Real", Zelenger 2022

"Superdeterminism" → Paul, 2026

≠

Zelinger
↳ universe is not Superdeterminism

Lec-03 05/01/26

Cryptography

- SHA
- RSA
- MD5

→ Cryptographic Hash Function (CHF)

- AES
- RC4
- ECDSA

→ Encryption function / CIPHER + Decryption function

The goal / definition of CHF Concept:

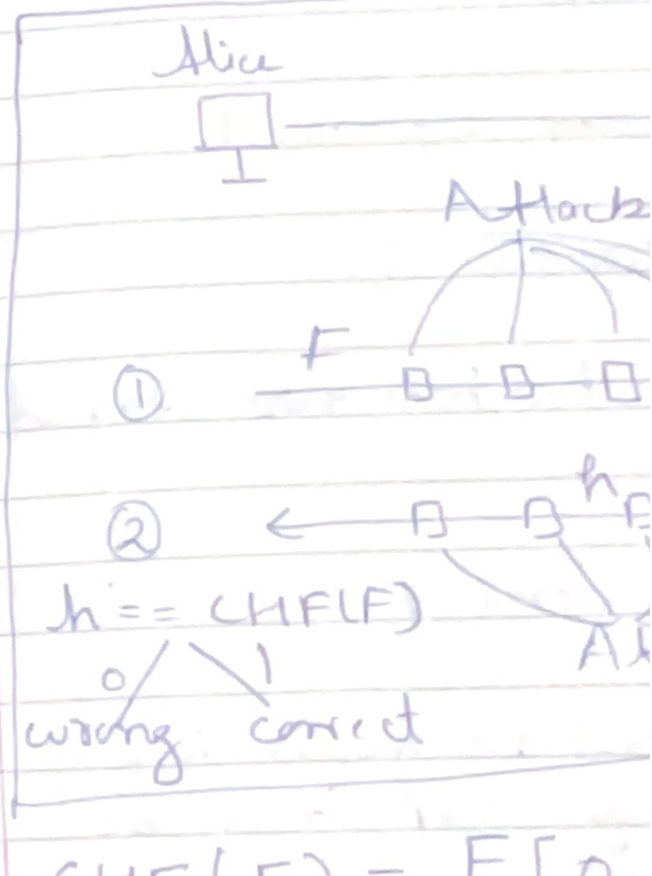
→ Input
→ Output
→ Properties → one-wayness
→ collision resistance

Input : $\{0,1\}^* = \{0,1\} \cup \{0,1\} \cup \{0,1\} \cup \ldots$

Output: ~~$\{0,1\}^*$~~ → $\{0,1\}^a$ → fixed length
- SHA-3 a=256

Meaning of a use-case

Protocol → Humans
→ Machine
→ Commun



CHF(F) = F[0.

Lec-04

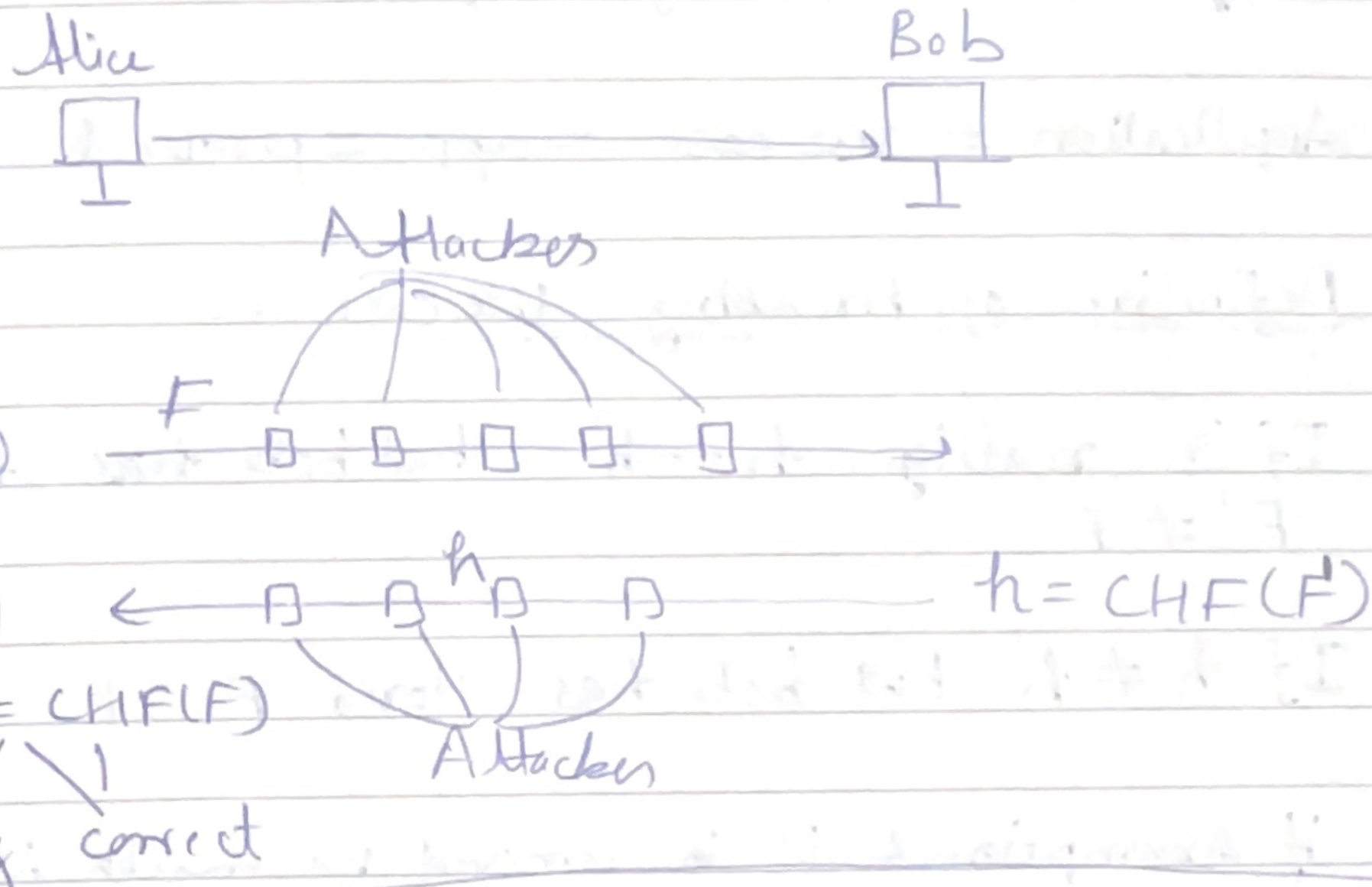
Usecase-1

Alice (F)

h = CHF(F)
if h == h'
then data auther
else data modifie

Meaning of a use-case of a CHF

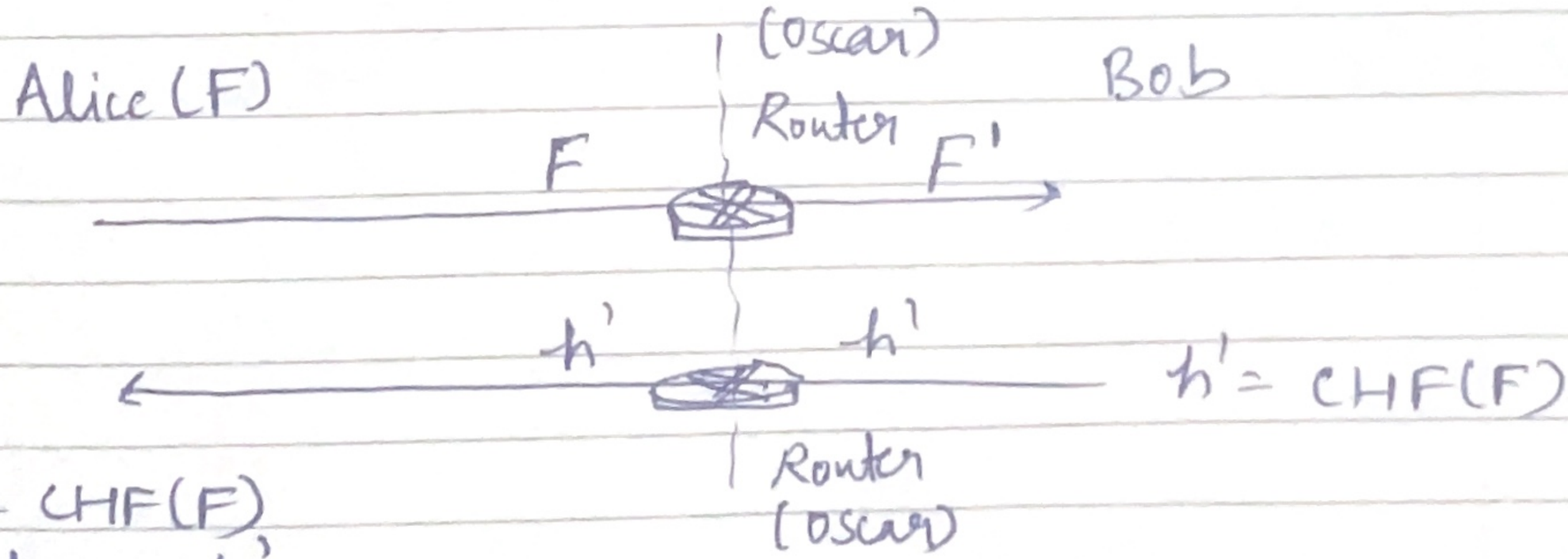
Protocol
- Humans
- Machines
- Communication channel



CHF(F) = F[0.....255]

Lec-04

Usecase-1



h = CHF(F)
if h == h'
then data authenticated
else data modified

Q without assumption-2, no matter how much effort we put in protocol, it can be attacked, Prove

Assumption

1. Bob does not modify F
2. Oscar does not modify h'
3. Alice & Bob can not forge their identities (System takes care of this)

Application = use-case = app = protocol

Definition of breaking Use-case-1

- If in reality $h == h'$ but bob has stored $F' \neq F$
- If $h \neq h'$ but bob has stored $F' = F$

Q. If Assumption-2 is is removed no matter how much effort we put to design CHF, it can be attacked. Prove

Ans

Design Attempt

① CHF: $F \longrightarrow F[0 \ldots 255]$ ✗

② CHF: $F \longrightarrow h$ (XOR)

where $h[0] = F[0] \oplus F[255]$

$h[1] = F[1] \oplus F[254]$

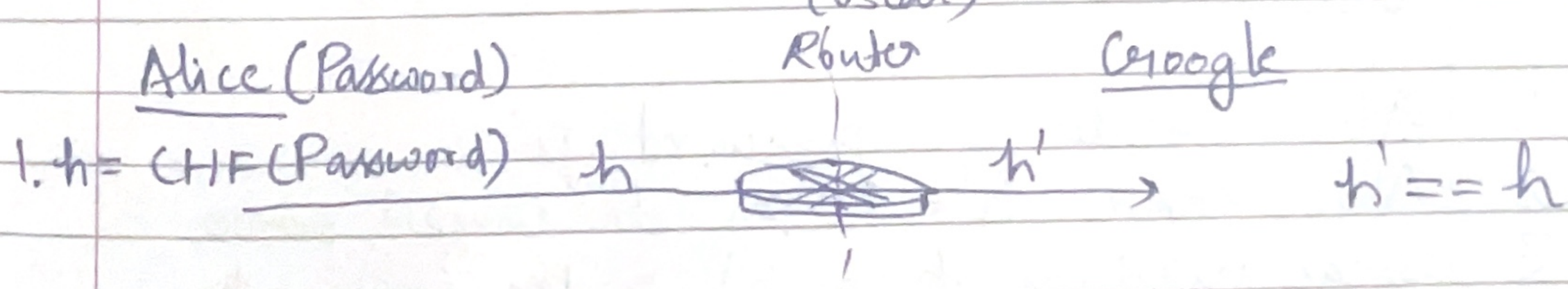
$h[2] = F[2] \oplus F[253]$

$h[255] = F[255] \oplus F[0]$

Q.1 If File size is fixed to 255 is this correct?

Ans

Use-Case-2 : Password authentication



Lec-5 08/01/26

Components of any protocol/usecase/application:
1) Humans
2) Local Machines
3) Communication channels

Use-case-2 (Password Generation)

Alice ☐ ↑ Bob Gmail Server

1. Create a password p
2. h := CHF(p)

h ⟷ store(h)

Assumptions:

1. Gmail Server is honest (not malicious)
2. Alice is honest
3. UserId are publicly known

What is the definition/meaning of "breaking a protocol":

1. Alice creates a password p)
2. Alice sends $h' = H(p)$ to Gmail server
3. Oscar modifies h' to h'' in the communication channel
4. Gmail receives $h'' \neq h'$ & stores it
   Table[Alice] = h''

Now oscar can send h'' & access Alices gmail account.

Another protocol to break

1. Oscar captures h
2. Oscar brute forces h to get p'
   s.t $CHF(p') = h$

Prevention / Mitigation

1. Make $CHF(\cdot)$ one way and collision resistant
   (Attack-2 is prevented)

Lec-6 12/01/26 (Not in Syllabus)

DSL353 Attendance System

Step-1 : Instructor clicks a photograph

Step-2 : Instructor shares the picture with the student.

Step-3 : Instructor marks attendance in the google sheet & share with the student.

Definition of attack
A student not present is marked present.
A student which is present is marked absent.

Assumptions:

1. Instructor is honest photographer
2. The server/database storage is honest and secure hackproof.
3. No student has a camera in the class.

Security

Institute attendance system

1. A student uses a academics controlled card-reader and id card (no processor) to send attendance data to academic server.

2. ~~Attendance~~ Academic server stores attendance data.

3. Academic shares attendance data with the student.

(Real Syllabus)

13/01/26 Lec-7

Usecase - 1 (Password generation)

Alice | Server

1. generate Password

2. password, username → 3. T[username] = password

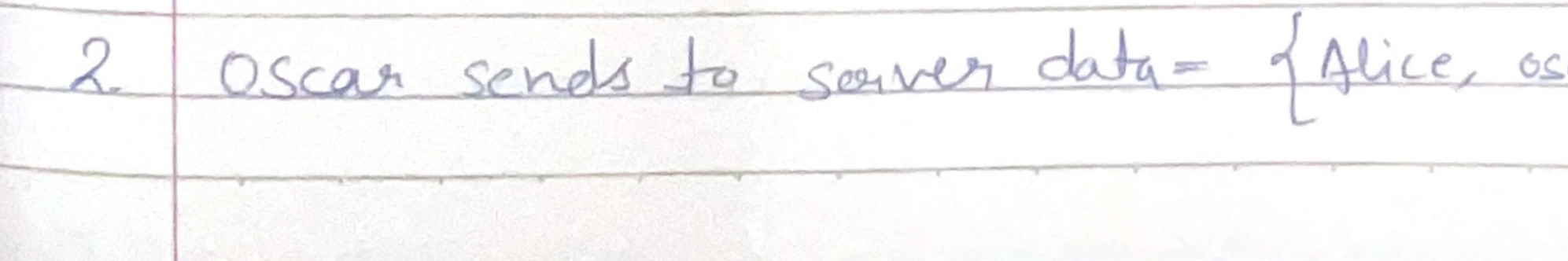
Usecase - 2 (Password authentication)

Alice (username, password)

1. username, password → 2. if T[username] == password then "verified" else "Invalid"

Assumption:

1. Alice & server are honest.



Attack usecase to break usecase-1

1. Oscar captures data = {username = Alice, password = Alice-password} & stops data from reaching the server.

2. Oscar sends to server data = {Alice, oscar-password}

Lec-8 15/01/26

Usecase: 15-01-2026

(San Fransisco) (Doha) (Delhi)

Alice (K) Oscar Bob (K)



Source Addr	Dest Addr	Data

Oscar can Capture

$E(K, x) = y$ $\longrightarrow y \longrightarrow$ $x = D(K, y)$

K → secret information
x → message

Secret encryption Algorithm

Symmetric key encryption

↓

Alice & Bob has same key

$E(K, x)$
$D(K, E(K, x)) = x$

* What security properties $E(\cdot,\cdot)$ & $D(\cdot,\cdot)$ would have so that usecase 15-01-2026 is secure?

Ans

$a \hat{} a = 0$

$0 \hat{} a = a$

Page No.:
Date:
YOUVA

Break-1 usecase 15-01-2026

1. Oscar captures y
2. Oscar has an algorithm, say A
   such that $A(y) \longrightarrow x'$
   such that $E(K, x') = y$

$E(K, x) = x^2 = y$	$E(K, x) = K \oplus x = y$
$D(K, y) = \sqrt{y}$	$D(K, y) = y \oplus K = x$
	$\rightarrow (K \oplus x) \oplus K = x$

Break-2 usecase 15-01-2026

1. Oscar ~~captures~~ sends x to Alice & asks her to encrypt x and send to him y.
2. Oscar captures y' (evening me)
3. Oscar determines (K)

Home-Assignment
Implement Standard encryption/decryption algorithm (AES-128) find an open-source implementation & try to encrypt your files.

Lec-9 23/01/26

Alice(secret key K) Bob(secret key k)

1. C := E(K, m)

m = D(K, C)

C →

* → Except secret key K all are publicly known
→ m = mail/message

1. Without a shared secret K Alice CANNOT send confidential message to Bob.

2. Without meeting in-person a secret-key can not be shared

Is there a fallacy?
YES

Encryption key = K
Decryption key = K

→ Decryption key is more important than the encryption key (if decryption key is leaked, msg will be leaked.)

→ Secret keys need not to be identical (but should be related).

$$K' = f(K)$$

↳ also known

PKC → only ensure confidentiality not authenticity

A ——→ B
1GB

RT rey small

M T W T F S S
Page No.:
Date:
YOUVA

Suppose:

→ Both Encryption key (K) and Decryption Key (K') are with Bob

→ It is hard to compute K' from K

↳ Bob can publish the encryption key (K)

⇒ K is no more a secret key

⇒ Alice can encrypt the message using publicly known encryption key

Reason: Since it is computationally hard to compute K' from K we can share K publicly.

Alice (encrypted key K)	Bob (encrypted key K decrypted key K')
C := E(K, m)	
——— C ———→	m = D(K', C)

→ Encrypted key := Public key
Decryption key := Private Key

eg. 1. MD5 (CHF) ——→ function
2. DES (SE (Symmetric encryption)) } definition
3. RSA (Public key encryption) }

||PKE|| ≃ 10^5 · ||SE|| (but sir said 10^4)

↳ Time complexity

4. DL Key established

next page

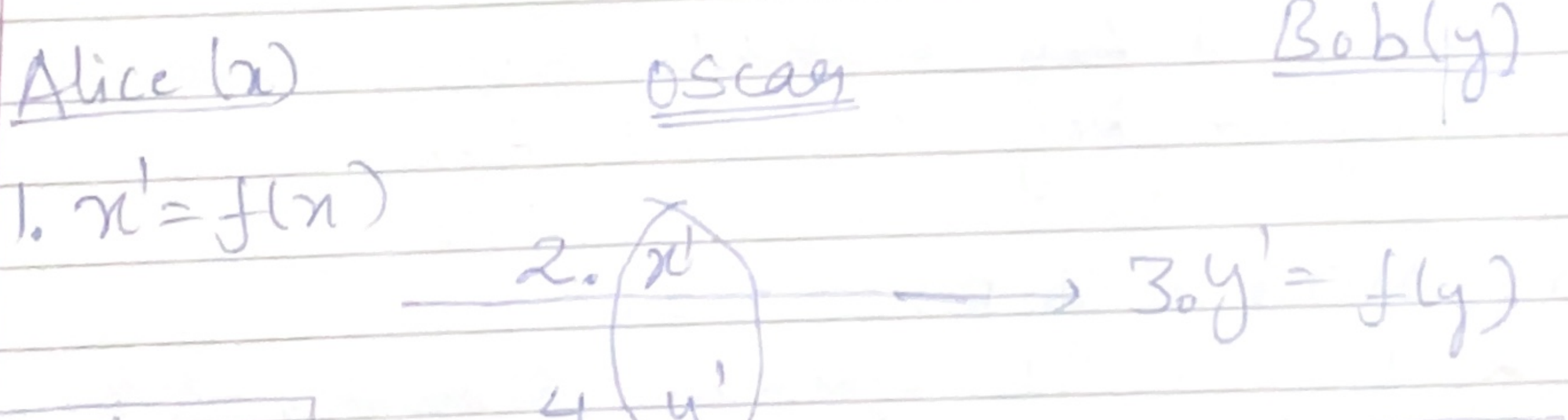
Q How Alice can send 1 GB of data to Bob using PKE with less computational time (SE is already ~~encrypted~~ implemented).

Soln Alice can send the symmetric encryption key to Bob by encrypting it with the public key

⇒ Now Alice can encrypt message using symmetric key & send that to bob.

Steps:

1. encrypt symmetric key using public key & Alice sends it to bob.
2. Now Alice encrypt message using SE & sends it to Bob.
3. Bob decrypts the SE key & then decrypts the message using the SE key.



$G_1(x, x', y') = Z$

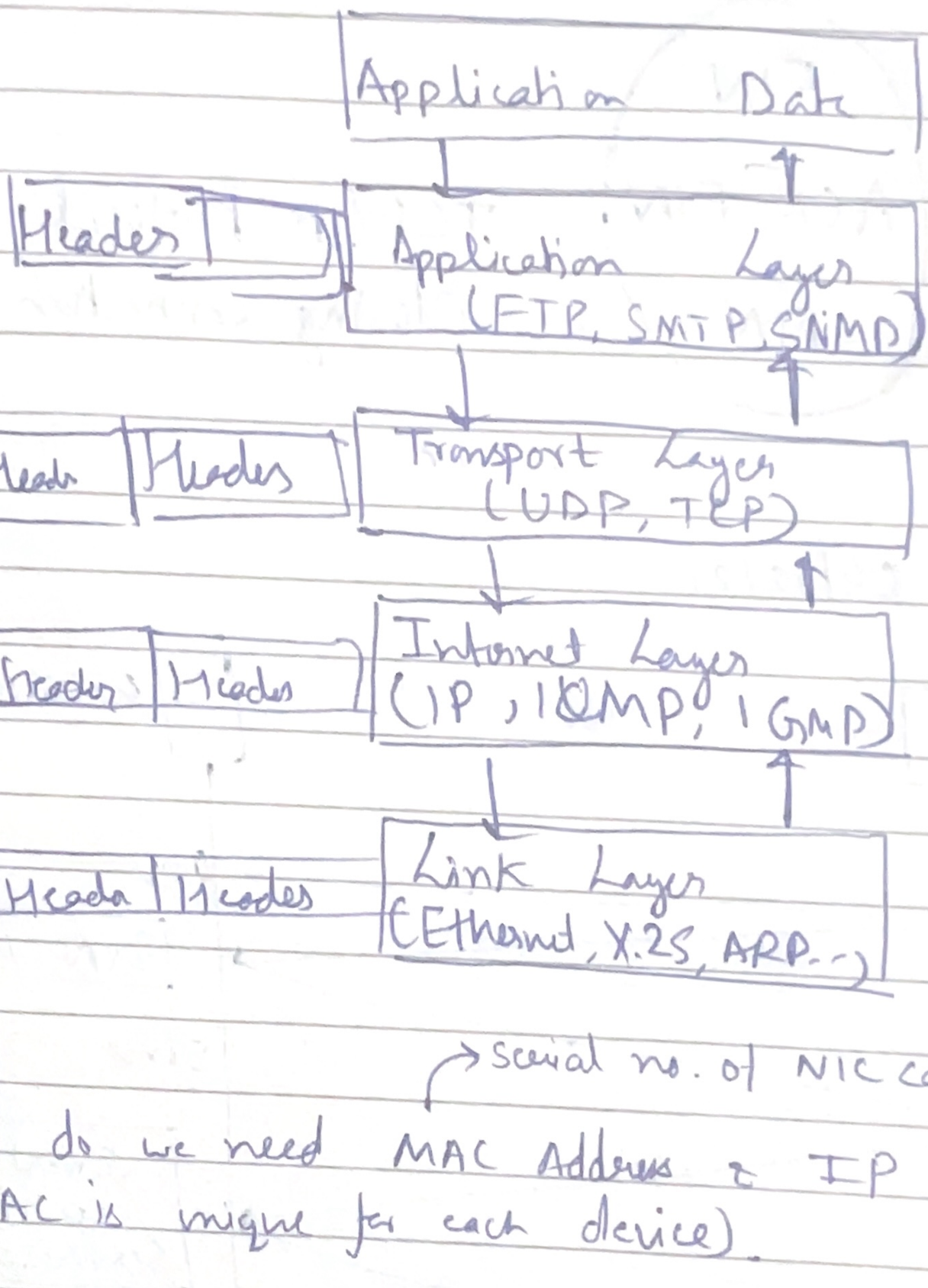
$G_1'(x', y', y) = Z$

Oscar can't know Z because he doesn't know x & y

Lec-10 3/02/26

1) Encryption — Symmetric
                — Public Key

2) Hash function



Q Why do we need MAC Address & IP Address (MAC is unique for each device).
↗ serial no. of NIC card

SE &
invt key
then
SE key.

Bob(y)

y' = f(y)

x', y', y

G'(x', y', y) =

aus the

SYN

ACK, SYN

SYN

TCP/IP Protocol
3-Way Handshake

Application Layer Protocol = http, https, VPN, DNS, DNS sec

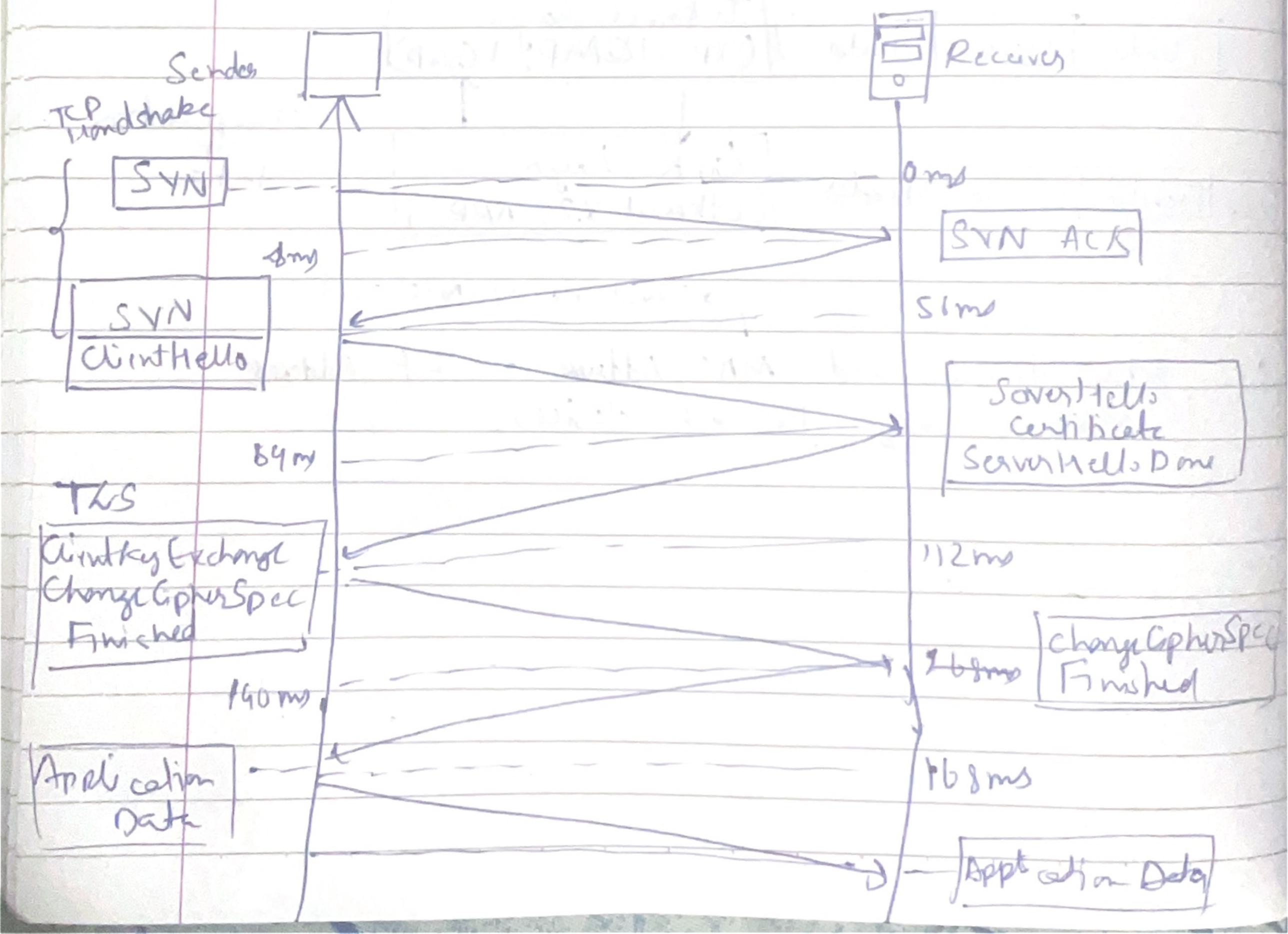
FIN

ACK, FIN

FIN

TCP/IP Protocol:
Closing connection

Lec-11 04/02/2(



Cryptographic algorithms

1) Symmetric encryption system (Eg DES, AES)
2) Public key encryption system (Eg RSA, Ecc)
3) Hash function (Eg SHA-1, MD5, MD4)
4) MAC = Message authentication code (Eg HMAC)

Client Hello → Client sends all the cryptographic algorithms it supports in order of preference

Server Hello → Server picks the best one both client & server supports

Server Cert. Done → Sends cert. along with public key

Client Key Exchange → Client creates a random no. (Symmetric key) & encrypts it using public key & sends back to server

Change Cipher Spec → Server uses the encrypted symmetric key & decrypts it using its private key to get the random no

Finished → Now both client & server have the random no (only valid for this session)

Cryptographic algorithms

1) Symmetric encryption system (Eg. DES, AES)
2) Public key Encryption system (Eg. RSA, ECC)
X3) Hash function (Eg. SHA-1/2/3, MD5, MD4)

4) MAC = Message authentication code (Eg. HMAC)

we will not use hash func^n in this usecase

ClientHello ⟶ client sends all the cryptographic algo available in it browser to server

ServerHello
Certificate
ServerHello Done ⟶ Server gets the list, both client & server accepts one one algo & server sends algo details along with public key.

ClientKeyExchange
ChangeCipherSpec
Finished ⟶ client creates a random no. (Symmetric key) & encrypt it using public key & sends back to server

ChangeCipherSpec
Finished ⟶ Server receives the encrypted msg & decrypt it using its private key to get the random no.

⇒ Now both client & server shares the random no (only valid for this session)

N ACK

ServerHello
Certificate
ServerHelloDone

12ms

ChangeCipherSpec
Finished

16.8ms

Application Data

Lec-12 05/02/26

Q. How client verifies that it is sending request to the verified & real servers?

Ans: Certificate

Public Key signature scheme (PKS scheme)

Two functions:

1. Sign()
2. verify()

Sign(P_r, m) $\longrightarrow$ σ (cryptographic signature) 300 bits

P_r ↑ Private Key

m ↑ message

Verify(P_u, σ, m) $\longrightarrow$ b $\in \{0,1\}$ (bit)

P_u ↑ Public Key

σ ↑ Signature

m ↑ msg

if b = 1 then the tuple (P_u, σ, m) are consistent ⇒ private key is also correct

Home Assignment: Analyze verify algorithm {Significance of b = 0/1}

(Simple) Usecase:

→ Not a good identity verification protocal,

(1) P_u is not verified.

(2) Oscar can catch σ, m & then pretend to be Alice.

↳ Use random challenge

V(Pu, Sign(Pr, m), r)

M T W T F S S
Page No.
Date:
YOUVA

(Simple) Usecase :

Alice (Pr) {Pu}

Bob

→ if challenge is not random then this is not safe
(random means → always generate fresh random no.)

r (random challenge) ←

σ := Sign(Pr, r)

σ ⟶ b := verify(Pu, σ, r)

if b=1 this is alice
else forgery

(σ doesn't changes)

→ Let's suppose random generator is bad ⇒ Oscar can get the σ once & then he can pretend to be Alice by sending σ again & again.

Q. How to check public key is correct (verified)?
→ Do physical verification once instead of verifying digitally everytime. (Certificate)

Lec-13 06/02/26

* Public Key Identity information
- IP Address
- MAC Address
- Port Address

} Identity documents (used to prove the identity)

→ Pu key & Pr key can't be generated by Certifying agency. Have to be generated by Applicant only.

M T W T F S S
Page No.:
Date:
YOUVA

Identity document generation protocal

- CERTIFICATE

Applicant — **CA** (Certifying agency)

1. Generates Pr/Pu

$\langle DOC \rangle$ = Pu, Name, Dob, Address, Father's Name →

Walk (not sending online) because someone could modify it.

$CERT_A = (\langle DOC \rangle, Sign(\langle DOC \rangle, Pr_{CA}))$ ← Private key of CA

Stores $\langle DOC \rangle$, $CERT_A$ ← $CERT_A$ (walk / not online) {can be / online also}

Alice (Pr_A, Pu_A) — **Bob**

Identity document verification

r →

$\sigma = Sign(Pr_A, r)$ →

→ but before this bob has to be verify the public key (Pu_A)

$b := verify(\sigma, r, Pu_A)$

Identity document Verification protocol

Bob

$b = verify(\langle DOC \rangle, Pu_{CA}, CERT_A)$

random challenge is not required for identity document verification.

Lec-14 10/02/26

→ certificate generation & verification

1) Public Key Signature Algorithm (PKS)
2) Symmetric Key Encryption / Decryption (SE)
3) Message Authentication Code (MAC)
4) Deffie Hellman Key Agreement protocal (DH)

Message Authentication Code (MAC)

Alice (k) {E(·), D(·)} Bob (k) {E(·), D(·)}

1. c = E(k, m)



Q What security property E(·) / D(·) achieves in the above protocal?

Ans Confidentiality (Now routers in between can't read the data)

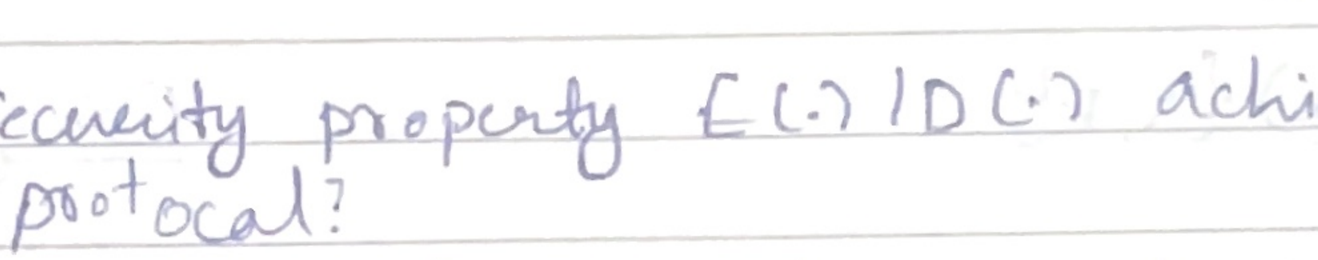
Data Authenticity ✗ (Someone can change the actual data → receiver will not receive the authentic data)

MAC

Alice (k)

1. C = E(k, m)
2. tag = MAC(k, m)

tag = H(m)
what if we use
Hash fn (EXAM ★★)



Bob (k)

3. m = D(k, c)
4. tag' = MAC(k, m)
5. tag == tag'

Discard Accept

Diagram: (Internet)

Diffe Hellman Protocal

Alice(x) {α, p}

Bobly) {α, p}

1. $\alpha^x \bmod p$

$\alpha^x \bmod p$ →

← $\alpha^y \bmod p$

2. $\alpha^y \bmod p$

3. $(\alpha^y)^x \bmod p = K$
$= \alpha^{xy} \bmod p$

4. $(\alpha^x)^y \bmod p = K$
$= \alpha^{xy} \bmod p$

Alice (x)

Oscar

Bobly)

1. 4x

4x →

2. 3y

← 3y

3. $\frac{3y}{3} \rightarrow y$

4. xy

5. $\frac{4x}{4} \rightarrow x$

5. xy

Assumption

1. Given a, α, p, it is hard to compute x
2. Giv b, α, p it is hard to compute y.

Lec-15 11/02/26

Q. Design a protocal where Alice identifies herself to Bob.

Ans